

Методичка: почтовая платформа Mailcow на Docker с доменом и диагностикой

Темы: Mailcow, SMTP, Submission, IMAP, MX, SPF, DKIM, DMARC, TLS, Docker Compose, mail queue, backup

ОС сервера: Debian Server 12/13 или Ubuntu Server 22.04/24.04 LTS

ОС клиента: Debian/Ubuntu или Windows 10/11

Среда: VirtualBox

Формат: самостоятельная практическая работа

Ориентировочное время: 6 академических часов

1. Что настроим

В этой работе собираем Linux-сервис или комплексную инфраструктуру. Команды выполняем на Debian/Ubuntu, результат проверяем локально, с клиента и через журналы.

Используем узлы: mail01, dns01, client01, zbx01.

Главная идея работы:

Mailcow считается развёрнутым не тогда, когда контейнеры запущены, а тогда, когда домен создан, ящики обмениваются письмами, DNS-записи понятны, очередь и журналы проверены, а восстановление контейнера не теряет данные volume.

2. Схема учебного стенда

Узел	Роль	ОС	Сетевые адаптеры	IP-адрес
lb01	балансировщик или входной узел	Debian/Ubuntu Server	NAT + внутренняя сеть	192.168.10.10 /24
web01	backend/web/mail/service	Debian/Ubuntu Server	NAT + внутренняя сеть	192.168.10.21 /24
web02	backend или дополнительный узел	Debian/Ubuntu Server	NAT + внутренняя сеть	192.168.10.22 /24
client01	клиент проверки	Debian/Ubuntu или Windows	внутренняя сеть	192.168.10.30 /24

```
client01 -> lb01/service entry -> web01/web02/mail/monitoring/backup
```

NAT используем для установки пакетов. Сервисы проверяем во внутренней сети. Имена добавляем через DNS или `/etc/hosts` и проверяем `getent hosts`.

3. Необходимые ресурсы и предварительные условия

Нужны:

- подготовленный сервер `mail01` с 4 vCPU, 8 ГБ RAM и 80+ ГБ диска;
- Docker и Compose;
- учебный домен `company.test` или отдельная лабораторная зона;
- записи A/MX через DNS или `/etc/hosts` для внутренней проверки;
- работа группами 2-3 человека допустима из-за ресурсов.

4. Подготовка виртуальных машин

```
hostnamectl  
ip -br address  
ip route  
resolvectl status  
systemctl --failed
```

Проверяем имена:

```
getent hosts lb01.company.test  
getent hosts web01.company.test  
getent hosts web02.company.test
```

Самопроверка этапа

- узлы включены;
- адреса и имена согласованы;
- NAT доступен для установки пакетов;
- снимки VM созданы.

Если результат отличается от ожидаемого, дальше пока не продолжаем. Сначала проверяем этот этап.

5. Адресный план или план ресурсов

Ресурс	Значение
Входной узел	lb01, 192.168.10.10
Backend 1	web01, 192.168.10.21
Backend 2	web02, 192.168.10.22
Клиент	client01, 192.168.10.30
Учебный домен	company.test

План работы:

- подготовить FQDN, время и DNS-записи;
- установить Docker/Compose;
- скачать Mailcow;
- создать домен и два mailbox;
- проверить webmail и отправку внутри стенда;
- проверить logs, queue, DKIM и контейнеры;
- смоделировать отказ контейнера и восстановление;
- определить backup-данные и мониторинг.

6. Исходная проверка

```
cat /etc/os-release
ip -br address
```

```
ip route
ss -lntup
systemctl --failed
df -h
```

Самопроверка этапа

- сеть работает;
- нужные порты свободны или понятны;
- свободного места достаточно;
- failed-службы разобраны.

7. Пошаговая настройка

7.1. Подготовка пакетов

```
sudo apt update
```

Если пакеты не скачиваются, проверяем NAT, DNS и маршрут.

7.2. Основная настройка

```
sudo apt install git docker.io docker-compose-plugin -y
sudo systemctl enable --now docker
sudo hostnamectl set-hostname mail01
getent hosts mail01.company.test
git clone https://github.com/mailcow/mailcow-dockerized.git /opt/mailcow-dockerized
cd /opt/mailcow-dockerized
./generate_config.sh
sudo docker compose pull
sudo docker compose up -d
sudo docker compose ps
```

В `generate_config.sh` указываем FQDN `mail01.company.test` для учебного

стенда. В web-интерфейсе создаём домен и два ящика, например

`admin@company.test` и `user@company.test`.

Самопроверка этапа

```
cd /opt/mailcow-dockerized
sudo docker compose ps
sudo docker compose logs --tail=80 postfix-mailcow
sudo docker compose logs --tail=80 dovecot-mailcow
ss -lntup | grep -E ':25|:587|:993|:443'
df -h
```

Через webmail отправляем письмо между двумя ящиками и проверяем получение. Затем перезапускаем один контейнер и проверяем восстановление.

Если результат отличается от ожидаемого, дальше пока не продолжаем. Сначала проверяем этот этап.

7.3. Восстановление или откат

Проверяем откат: возвращаем backend, восстанавливаем контейнер, проверяем бэкапы или возвращаем конфигурацию из `.bak`. Для сложных сервисов сначала восстанавливаем тестовый объект, а не весь сервер.

8. Проверка итогового результата

```
cd /opt/mailcow-dockerized
sudo docker compose ps
sudo docker compose logs --tail=80 postfix-mailcow
sudo docker compose logs --tail=80 dovecot-mailcow
ss -lntup | grep -E ':25|:587|:993|:443'
df -h
```

Через webmail отправляем письмо между двумя ящиками и проверяем получение. Затем перезапускаем один контейнер и проверяем восстановление.

Границы результата:

- учебная настройка не заменяет промышленную HA-архитектуру;
- публичная почта требует реального DNS, PTR, репутации IP и TLS;

- тестовые пароли, DKIM-ключи и секреты нельзя публиковать;
- сервис считается готовым только после клиентской проверки и журналов.

9. Диагностика типовых ошибок

Симптом	Вероятная причина	Проверка	Исправление
Web UI недоступен	Контейнер nginx-mailcow не поднят или порт занят	<code>docker compose ps, ss -lntup</code>	Проверить порты и контейнер
Письма не уходят	DNS/MX/FQDN или очередь	postfix logs, mail queue	Исправить DNS и посмотреть queue
IMAP не работает	Dovecot container или порт	dovecot logs, <code>ss -lntup</code>	Перезапустить контейнер и проверить volume
Мало ресурсов	RAM/диск недостаточны	<code>free -h, df -h, docker stats</code>	Увеличить ресурсы или использовать готовый сервер

Порт занят	Другая служба уже слушает	<code>ss -lntup</code>	Освободить порт или изменить конфигурацию
Клиент не подключается	DNS, firewall или служба	<code>getent hosts, nc - vz, journalctl</code>	Исправить имя, порт или службу
Диск заполнен	Логи, mail queue, Docker volumes или backup	<code>df -h, du - xhd1</code>	Очистить или расширить хранилище по плану

10. Итоговая самостоятельная проверка

- ☐ Стенд соответствует схеме.
- ☐ Имена и адреса согласованы.
- ☐ Пакеты или контейнеры установлены.
- ☐ Основной сервис запущен.
- ☐ Порт слушается.
- ☐ Клиентская проверка проходит.
- ☐ Отказ или восстановление проверены.
- ☐ В журналах нет необъяснённых ошибок.

- [] Одна типовая ошибка найдена и исправлена.

11. Что приложить к отчёту

К отчёту прикладываем схему, адресный план, конфигурацию, вывод проверки службы/контейнеров, клиентский результат, журнал, результат отказового теста или восстановления, описание исправленной ошибки.

12. Контрольные вопросы

1. Какую задачу решает технология этой лекции?
2. Какие зависимости нужно проверить до запуска сервиса?
3. Какая команда подтверждает основной результат?
4. Как проверить сервис с клиента?
5. Что искать в журналах?
6. Как выполняется восстановление?
7. Какие ограничения есть у учебного стенда?
8. Что обязательно документировать?